AKADEMIA NAUK STOSOWANYCH W ELBLĄGU

Instytut Informatyki Stosowanej im. Krzysztofa Brzeskiego

System Obsługi Praktyk Zawodowych

Dokumentacja Techniczna i Funkcjonalna

Spis treści

Spis treści 2
1. Wstęp i cel projektu 3
1.1. Cel systemu 3
1.2. Zakres dokumentacji 3
2. Architektura systemu 4
2.1. Stos technologiczny 4
2.2. Struktura projektu 4
2.3. Obsługa danych — faza I (JSON) 5
3. Role i uprawnienia użytkowników 6
3.1. Definicje ról 6
3.2. Implementacja kontroli dostępu 6
3.3. Macierz uprawnień do załączników 6
4. Przepływ procesu praktyki zawodowej 8
4.1. Ścieżka standardowa 8
4.2. Ścieżka alternatywna — uznanie pracy zawodowej / stażu 8
5. Opis modułów i formularzy 10
5.1. Moduł autoryzacji — blueprint: auth 10
5.2. Moduł studenta — blueprint: student 10
5.3. Formularz Zał. 6 — Dziennik praktyki (dynamiczna tabela) 10
5.4. Formularz Zał. 4 — Potwierdzenie efektów uczenia się 11
5.5. Moduł UOPZ — blueprint: uopz 11
5.6. Moduł ZOPZ — blueprint: zopz 12
5.7. Moduł dziekanatu — blueprint: dziekanat 12
6. Statusy praktyki w systemie 13
7. Wymagania niefunkcjonalne 14
7.1. Bezpieczeństwo 14
7.2. Użyteczność 14
7.3. Środowisko uruchomieniowe 14
7.4. Uruchomienie projektu 14
8. Status projektu i plan prac 15

1. Wstęp i cel projektu

Niniejszy dokument stanowi dokumentację techniczną i funkcjonalną systemu obsługi praktyk zawodowych dla kierunku Informatyka w Instytucie Informatyki Stosowanej im. Krzysztofa Brzeskiego ANS w Elblągu.

Obecny proces obsługi praktyk opiera się na 11 papierowych załącznikach, które muszą być ręcznie drukowane, wypełniane, podpisywane i dostarczane do odpowiednich osób. Generuje to szereg problemów: zagubione dokumenty, brak wglądu w status praktyki, konieczność fizycznej obecności w dziekanacie, brak historii zmian.

1.1. Cel systemu

System ma na celu pełną digitalizację i automatyzację obiegu dokumentów związanych z praktykami zawodowymi. Kluczowe założenia:

- Zastąpienie papierowych załączników cyfrowymi formularzami wypełnianymi przez przeglądarkę
- Kontrola dostępu oparta na rolach — każdy użytkownik widzi tylko to, do czego ma uprawnienia
- Śledzenie statusu praktyki w czasie rzeczywistym
- Możliwość generowania gotowych dokumentów PDF na podstawie wprowadzonych danych
- Powiadamianie uczestników procesu o wymaganych działaniach przez e-mail

1.2. Zakres dokumentacji

Dokument obejmuje: opis architektury systemu, specyfikację ról i uprawnień, opis przepływu procesu, opis wszystkich obsługiwanych formularzy oraz szczegóły implementacyjne dla poszczególnych modułów aplikacji Flask.

2. Architektura systemu

2.1. Stos technologiczny

System zbudowany jest w architekturze klient-serwer. Backend oparty jest na frameworku Flask (Python), komunikacja z przeglądarką odbywa się przez HTTP, a widoki renderowane są po stronie serwera przy użyciu szablonów Jinja2.

Warstwa	Technologia	Opis zastosowania
Backend	Python 3.11+, Flask	Główna logika aplikacji, routing, przetwarzanie formularzy
Autoryzacja	Flask-Login	Zarządzanie sesją, ochrona tras, tożsamość użytkownika
Formularze	Flask-WTF, WTForms	Walidacja danych, ochrona CSRF, dynamiczne pola listowe
Szablony	Jinja2	Renderowanie HTML po stronie serwera, pętle i warunki
Frontend	Bootstrap 5, JavaScript	Responsywny interfejs, dynamiczne wiersze tabel (addRow/deleteRow)
Baza danych (faza I)	JSON (load_data / save_data)	Zapis formularzy do plików .json per student
Baza danych (faza II)	PostgreSQL / SQLite	Docelowe środowisko produkcyjne (planowane)
Generowanie PDF	WeasyPrint / ReportLab	Eksport wypełnionych formularzy do PDF
Powiadomienia	Flask-Mail	E-mail do opiekunów, studentów, dziekanatu

2.2. Struktura projektu

Projekt podzielony jest na blueprinty Flask — każda rola posiada własny moduł z trasami i formularzami. Dane przechowywane są w plikach JSON z podziałem na typ załącznika i numer albumu studenta.

Struktura katalogów

```
praktyki-system/
├── app/
│   ├── __init__.py          # Fabryka aplikacji Flask
│   ├── config.py            # Konfiguracja środowisk (dev/prod)
│   ├── blueprints/
│   │   ├── auth/            # Logowanie, wylogowanie, reset hasła
│   │   ├── student/         # Widoki i formularze studenta
│   │   ├── uopz/            # Uczelniany opiekun praktyk
│   │   ├── zopz/            # Zakładowy opiekun praktyk
```

```
│   │   ├── dziekanat/        # Dziekanat / Dyrektor Instytutu
│   │   ├── admin/            # Zarządzanie użytkownikami i rolami
│   │   └── dokumenty/        # Generowanie i pobieranie PDF
│   ├── forms/                # Formularze Flask-WTF / WTForms
│   ├── templates/            # Szablony Jinja2
│   └── static/               # CSS, JS, Bootstrap
├── data/                     # Pliki JSON (faza I)
│   ├── users.json
│   ├── praktyki.json
│   ├── zal4_efekty/          # Zał. 4 per student
│   └── zal6_dziennik/        # Zał. 6 per student
├── dokumentacja/
├── requirements.txt
└── run.py
```

2.3. Obsługa danych — faza I (JSON)

W pierwszej fazie projektu dane przechowywane są w plikach JSON. Każdy formularz zapisywany jest do osobnego pliku powiązanego z numerem albumu studenta. Poniżej przedstawiono funkcje pomocnicze używane we wszystkich trasach:

Funkcje pomocnicze — app/utils.py

```
import json, os

def load_data(path):
    if os.path.exists(path):
        with open(path, 'r', encoding='utf-8') as f:
            return json.load(f)
    return []

def save_data(path, data):
    os.makedirs(os.path.dirname(path), exist_ok=True)
    with open(path, 'w', encoding='utf-8') as f:
        json.dump(data, f, indent=4, ensure_ascii=False)
```

3. Role i uprawnienia użytkowników

System obsługuje pięć ról. Dostęp do dokumentu jest przyznawany na poziomie widoku (podgląd) lub edycji (możliwość zapisu/podpisu). Użytkownik wymieniony w dokumencie zawsze może go zobaczyć, ale nie zawsze edytować.

3.1. Definicje ról

Rola	Identyfikator	Opis
Student	student	Osoba odbywająca praktykę. Wypełnia wnioski, prowadzi dziennik, pisze sprawozdanie i ankietę.
Uczelniany Opiekun Praktyk	uopz	Pracownik uczelni. Wydaje skierowania, nadzoruje przebieg, przeprowadza egzamin, wpisuje ocenę do USOS.
Zakładowy Opiekun Praktyk	zopz	Pracownik zakładu pracy. Potwierdza wpisy w dzienniku, podpisuje kartę praktyki i sprawozdanie.
Dziekanat / Dyrektor	dziekanat	Podpisuje porozumienia, przyjmuje oświadczenia, podejmuje decyzje o uznaniu efektów uczenia się.
Administrator	admin	Pełny dostęp do systemu. Zarządza użytkownikami, rolami i konfiguracją.

3.2. Implementacja kontroli dostępu

Kontrola dostępu realizowana jest przez dekoratory Flask-Login oraz własny dekorator @role_required. Każda trasa jest chroniona — nieautoryzowany dostęp zwraca błąd 403.

Dekorator @role_required — app/auth/decorators.py

```
from functools import wraps
from flask import abort
from flask_login import current_user

def role_required(*roles):
    def decorator(f):
        @wraps(f)
        def decorated(*args, **kwargs):
            if not current_user.is_authenticated:
                abort(401)
            if current_user.rola not in roles:
                abort(403)
            return f(*args, **kwargs)
        return decorated
    return decorator
```

3.3. Macierz uprawnień do załączników

Poniższa tabela przedstawia pełną macierz dostępu do każdego z 11 załączników. Symbol V oznacza podgląd (view), symbol E oznacza edycję lub możliwość podpisania dokumentu.

Załącznik	Student	UOPZ	ZOPZ	Dziekanat	Admin
Zał. 1 – Porozumienie z zakładem pracy	V	V	—	E / Podpis	E
Zał. 2 – Program praktyki	V	E	V	V	E
Zał. 2a – Program i harmonogram	V	E	V	V	E
Zał. 3 – Karta praktyki + Skierowanie	V	E (skierowanie)	E (opinia)	V	E
Zał. 4 – Potwierdzenie efektów uczenia się	V	V	E	V	E
Zał. 4a – Ocena wniosku studenta	—	E	—	E (decyzja)	E
Zał. 4b – Wniosek studenta	E	V	—	V	E
Zał. 5 – Kwestionariusz ankiety	E	V	—	—	V
Zał. 6 – Dziennik praktyki	E (wpisy)	V	E (podpis)	V	E
Zał. 7 / 7a – Sprawozdanie	E	V	E (podpis)	V	E
Zał. 8 – Protokół egzaminu	V (własny)	E	—	E	E
Zał. 9 – Oświadczenie instytucji	V	V	—	E	E

4. Przepływ procesu praktyki zawodowej

4.1. Ścieżka standardowa

Poniżej przedstawiono pełny przebieg procesu praktyki zawodowej realizowanej w oparciu o porozumienie z zakładem pracy. Każdy krok oznaczony jest odpowiednim załącznikiem i rolą odpowiedzialną za jego realizację.

Krok	Działanie	Rola	Załącznik	Status w systemie
1	Student wyszukuje zakład pracy i uzyskuje potwierdzenie przyjęcia	Student	Zał. 9	OCZEKUJE_NA_ZAL9
2	Student dostarcza Zał. 9 do dziekanatu	Dziekanat	Zał. 9	ZAL9_PRZYJETY
3	Uczelnia podpisuje porozumienie z zakładem i wysyła Regulamin + Zał. 2	Dziekanat	Zał. 1, 2	POROZUMIENIE_PODPISANE
4	UOPZ, ZOPZ i student uzgadniają program i harmonogram	UOPZ, ZOPZ, Student	Zał. 2a	PROGRAM_UZGODNIONY
5	UOPZ wydaje skierowanie na praktykę	UOPZ	Zał. 3	SKIEROWANIE_WYDANE
6	Student rozpoczyna praktykę (960h / 120 dni / 6 miesięcy)	Student	—	PRAKTYKA_W_TOKU
7	Student codziennie wypełnia dziennik; ZOPZ potwierdza wpisy	Student, ZOPZ	Zał. 6	PRAKTYKA_W_TOKU
8	UOPZ przeprowadza minimum 1 hospitację w zakładzie	UOPZ	—	PRAKTYKA_W_TOKU
9	Do 7 dni po zakończeniu student składa komplet dokumentów	Student	Zał. 3, 4, 6, 7, 5	DOKUMENTY_ZLOZONE
10	Egzamin ustny przed komisją egzaminacyjną	UOPZ + Komisja	Zał. 8	EGZAMIN
11	Komisja sporządza protokół zaliczenia	UOPZ + Komisja	Zał. 8	PROTOKOL_SPORZADZONY
12	UOPZ wpisuje ocenę do systemu USOS	UOPZ	—	ZALICZONA

4.2. Ścieżka alternatywna — uznanie pracy zawodowej / stażu

Student, który pracuje zawodowo lub prowadzi działalność gospodarczą, może wystąpić z wnioskiem o zaliczenie praktyki na podstawie dotychczasowego doświadczenia. Wymagane jest, aby zakres obowiązków był zgodny z programem praktyki zawodowej.

1. Student składa wniosek — Zał. 4b (Wniosek o zaliczenie efektów uczenia się)
2. Do wniosku dołącza: zaświadczenie o zatrudnieniu, opis stanowiska pracy, zaświadczenie z CEIDG lub odpis KRS
3. Komisja ds. praktyk dokonuje merytorycznej oceny wniosku — Zał. 4a
4. Ostateczną decyzję podejmuje Dyrektor Instytutu
5. W przypadku decyzji pozytywnej: UOPZ wpisuje zaliczenie do USOS
6. W przypadku decyzji negatywnej: student realizuje praktykę ścieżką standardową

5. Opis modułów i formularzy

5.1. Moduł autoryzacji — blueprint: auth

Moduł odpowiada za logowanie, wylogowanie i resetowanie hasła. Każdy użytkownik posiada konto z przypisaną rolą. Hasła przechowywane są w formie skrótu (hash).

Trasa	Metoda	Opis
/login	GET, POST	Formularz logowania — email + hasło. Po poprawnym logowaniu przekierowanie według roli.
/logout	GET	Wylogowanie użytkownika, czyszczenie sesji.
/reset-hasla	GET, POST	Formularz do resetowania hasła przez e-mail.

5.2. Moduł studenta — blueprint: student

Panel studenta umożliwia podgląd statusu praktyki, wypełnianie przypisanych formularzy oraz śledzenie postępu dokumentacji.

Trasa	Metoda	Opis
/student/dashboard	GET	Strona główna — status praktyki, lista dokumentów do uzupełnienia.
/student/zal4b	GET, POST	Formularz wniosku o zaliczenie pracy zawodowej (Zał. 4b).
/student/zal5	GET, POST	Kwestionariusz ankiety oceniający przebieg praktyki (Zał. 5).
/student/zal6	GET, POST	Dziennik praktyki — dynamiczna tabela wierszy z opisem prac (Zał. 6).
/student/zal7	GET, POST	Sprawozdanie z praktyki zawodowej (Zał. 7 lub 7a).
/student/dokumenty	GET	Lista wszystkich dokumentów studenta z ich statusami.

5.3. Formularz Zał. 6 — Dziennik praktyki (dynamiczna tabela)

Dziennik praktyki ma formę tabeli, w której każdy wiersz odpowiada jednemu dniu pracy. Student może dodawać wiersze dynamicznie bez przeładowania strony. ZOPZ potwierdza każdy wpis cyfrowym podpisem.

Struktura danych zapisywana do pliku data/zal6_dziennik/{nr_albumu}.json:

Przykład struktury JSON — Zał. 6

```
{
  "nr_albumu": "12345",
  "miejsce_praktyki": "Nazwa Firmy Sp. z o.o.",
```

```
  "data_rozpoczecia": "2024-10-01",
  "data_zakonczenia": "2025-02-14",
  "wpisy": [
    {
      "dzien": 1,
      "data": "2024-10-01",
      "opis": "Zapoznanie z infrastrukturą sieci...",
      "nr_efektu": "EK_01, EK_03",
      "potwierdzenie_zopz": false
    }
  ]
}
```

Obsługa dynamicznych wierszy po stronie frontendu realizowana jest w czystym JavaScript. Formularz używa pól z nawiasami kwadratowymi w atrybucie name (np. name="opis[]"), co pozwala Flaskowi odebrać dane jako listę przez request.form.getlist().

5.4. Formularz Zał. 4 — Potwierdzenie efektów uczenia się

Formularz wypełniany przez ZOPZ po zakończeniu praktyki. Zawiera listę efektów uczenia się przypisanych do praktyki zawodowej — ZOPZ zaznacza przy każdym efekcie, czy student go osiągnął.

Przykład struktury JSON — Zał. 4

```
{
  "nr_albumu": "12345",
  "ilosc_godzin": 960,
  "efekty": [
    { "kod": "EK_01", "opis": "Ma wiedzę o realizacji zadań inżynierskich...",
"uzyskany": true },
    { "kod": "EK_02", "opis": "Potrafi projektować systemy informatyczne...",
"uzyskany": true }
  ],
  "podpis_zopz": "Jan Kowalski",
  "data_podpisu": "2025-02-15"
}
```

5.5. Moduł UOPZ — blueprint: uopz

Trasa	Metoda	Opis
/uopz/studenci	GET	Lista wszystkich studentów z aktualnym statusem praktyki.
/uopz/skierowanie/<id>	GET, POST	Wydanie skierowania na praktykę (Zał. 3).
/uopz/program/<id>	GET, POST	Edycja programu i harmonogramu praktyki (Zał. 2a).

Trasa	Metoda	Opis
/uopz/hospitacja/<id>	GET, POST	Rejestracja hospitacji w zakładzie pracy.
/uopz/egzamin/<id>	GET, POST	Wystawienie oceny i sporządzenie protokołu (Zał. 8).
/uopz/efekty/<id>	GET	Podgląd i uznanie efektów uczenia się (Zał. 4).

5.6. Moduł ZOPZ — blueprint: zopz

Trasa	Metoda	Opis
/zopz/dziennik/<id>	GET, POST	Podgląd dziennika studenta — potwierdzanie wpisów (Zał. 6).
/zopz/efekty/<id>	GET, POST	Wypełnienie potwierdzenia efektów uczenia się (Zał. 4).
/zopz/karta/<id>	GET, POST	Podpisanie karty praktyki i wystawienie opinii (Zał. 3).
/zopz/sprawozdanie/<id>	GET, POST	Podpisanie sprawozdania studenta (Zał. 7 / 7a).

5.7. Moduł dziekanatu — blueprint: dziekanat

Trasa	Metoda	Opis
/dziekanat/porozumienia	GET	Lista porozumień z zakładami pracy — podgląd i podpisanie (Zał. 1).
/dziekanat/oswiadczenia	GET, POST	Przyjęcie oświadczenia instytucji dostarczonego przez studenta (Zał. 9).
/dziekanat/wnioski	GET	Lista wniosków o zaliczenie pracy zawodowej (Zał. 4b + 4a).
/dziekanat/komisja	GET, POST	Zarządzanie składem komisji egzaminacyjnej.

6. Statusy praktyki w systemie

Każda praktyka w systemie posiada aktualny status odzwierciedlający etap procesu. Status zmienia się automatycznie po zatwierdzeniu dokumentów przez uprawnione osoby.

Status	Opis	Przejście do następnego
OCZEKUJE_NA_ZAL9	Student zarejestrowany, brak oświadczenia instytucji	Student dostarcza Zał. 9
ZAL9_PRZYJETY	Dziekanat przyjął Zał. 9	Dziekanat podpisuje Zał. 1
POROZUMIENIE_PODPISANE	Porozumienie z zakładem podpisane	Uzgodnienie programu (Zał. 2a)
PROGRAM_UZGODNIONY	Program i harmonogram zatwierdzony	UOPZ wydaje skierowanie
SKIEROWANIE_WYDANE	Skierowanie wystawione przez UOPZ	Student rozpoczyna praktykę
PRAKTYKA_W_TOKU	Praktyka trwa — student prowadzi dziennik	Student składa dokumenty po zakończeniu
DOKUMENTY_ZLOZONE	Komplet dokumentów złożony do UOPZ	Komisja przeprowadza egzamin
EGZAMIN	Egzamin ustny przed komisją w toku	Komisja sporządza protokół
PROTOKOL_SPORZADZONY	Protokół zaliczenia podpisany	UOPZ wpisuje ocenę do USOS
ZALICZONA	Praktyka zaliczona, ocena wpisana do USOS	— (stan końcowy)
ODRZUCONA	Wniosek o zaliczenie pracy odrzucony przez Dyrektora	Student wraca do ścieżki standardowej
PRZEDLUZONA	Praktyka przedłużona na wniosek studenta (max 1 miesiąc)	Powrót do PRAKTYKA_W_TOKU

7. Wymagania niefunkcjonalne

7.1. Bezpieczeństwo

- Wszystkie hasła przechowywane jako skróty (np. bcrypt przez Flask-Bcrypt)
- Każdy formularz chroniony tokenem CSRF (Flask-WTF)
- Wszystkie trasy wymagające logowania chronione przez @login_required
- Dostęp do zasobów innych użytkowników blokowany przez @role_required
- Pliki JSON i PDF niedostępne bezpośrednio przez URL — serwowane przez Flask

7.2. Użyteczność

- Interfejs responsywny — działanie na komputerach i urządzeniach mobilnych (Bootstrap 5)
- Dynamiczne dodawanie wierszy w tabelach bez przeładowania strony (JavaScript)
- Czytelny panel statusu praktyki dla każdej roli
- Powiadomienia e-mail przy kluczowych zmianach statusu

7.3. Środowisko uruchomieniowe

Wymaganie	Minimalna wersja
Python	3.11+
Flask	3.0+
Node.js (tylko do budowania frontendu)	18+
System operacyjny	Linux / Windows / macOS

7.4. Uruchomienie projektu

7. Sklonuj repozytorium: git clone https://github.com/...
8. Utwórz wirtualne środowisko: python -m venv venv
9. Aktywuj środowisko: source venv/bin/activate (Linux) lub venv\Scripts\activate (Windows)
10. Zainstaluj zależności: pip install -r requirements.txt
11. Uruchom aplikację: flask run

Aplikacja dostępna pod adresem: http://127.0.0.1:5000

8. Status projektu i plan prac

Funkcjonalność	Status	Priorytet
Diagramy architektoniczne i dokumentacja	Gotowe	Wysoki
Struktura projektu Flask i konfiguracja	W trakcie	Wysoki
System logowania i zarządzanie rolami	Planowane	Wysoki
Formularz Zał. 4 — Potwierdzenie efektów	Planowane	Wysoki
Formularz Zał. 6 — Dziennik (dyn. tabela)	Planowane	Wysoki
Pozostałe załączniki (1, 2, 3, 5, 7, 8, 9)	Planowane	Średni
Generowanie PDF z formularzy	Planowane	Średni
Powiadomienia e-mail	Planowane	Niski
Migracja z JSON do bazy danych (faza II)	Planowane	Niski